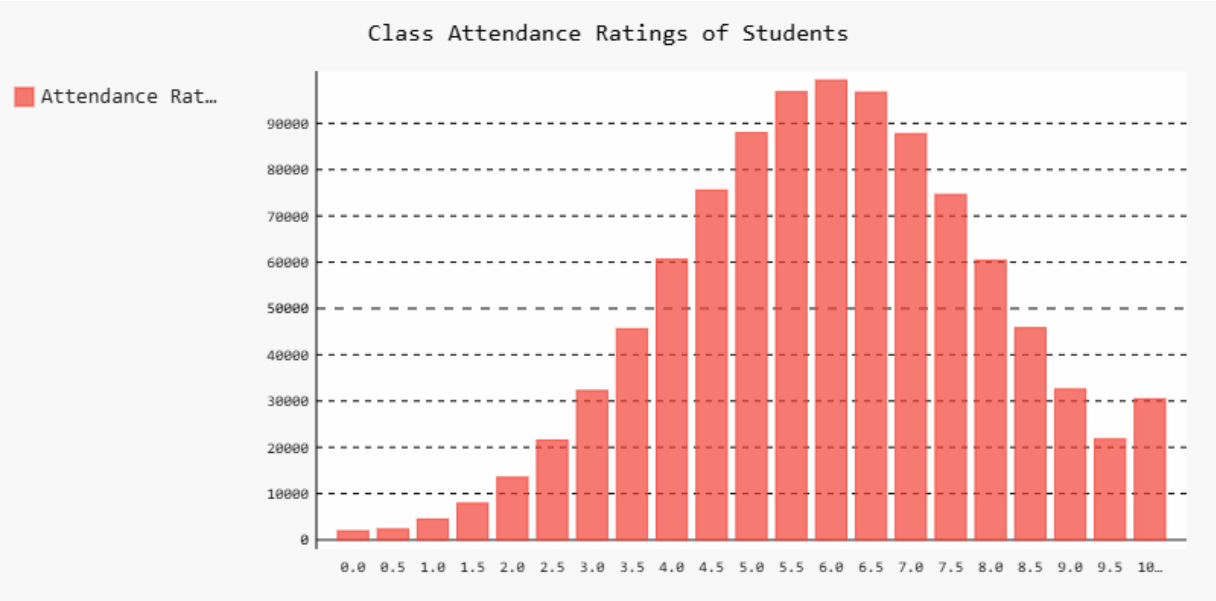
Results and Summary:

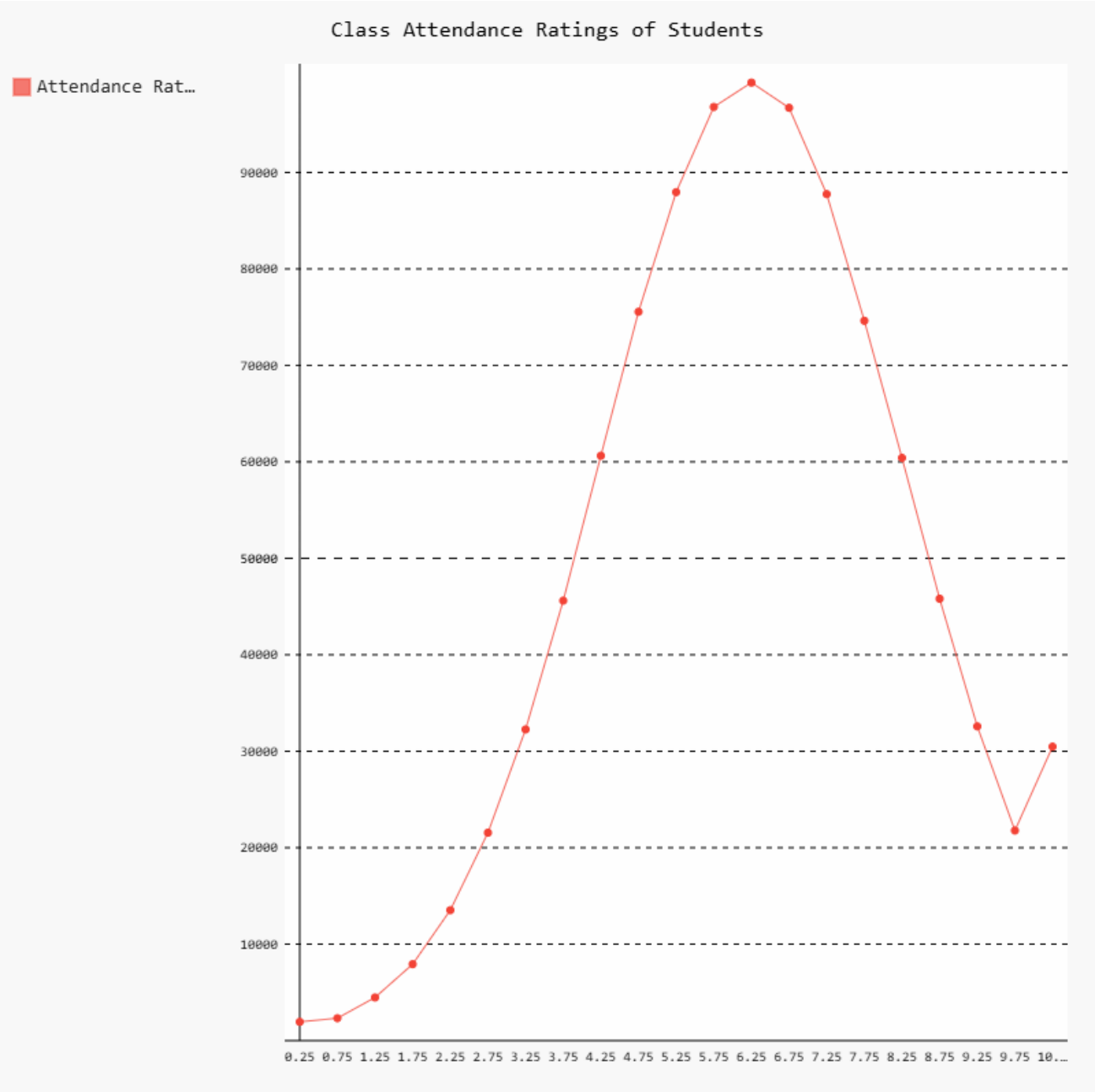
Frequency Table:

Attendance Rating	Frequency
0.0	1947
0.5	2326
1.0	4475
1.5	7927
2.0	13526
2.5	21556
3.0	32263
3.5	45611
4.0	60630
4.5	75556
5.0	87954
5.5	96787
6.0	99316
6.5	96699
7.0	87753
7.5	74623
8.0	60410
8.5	45808
9.0	32578
9.5	21780
10.0	30475

Bar Diagram:



Frequency Polygon:

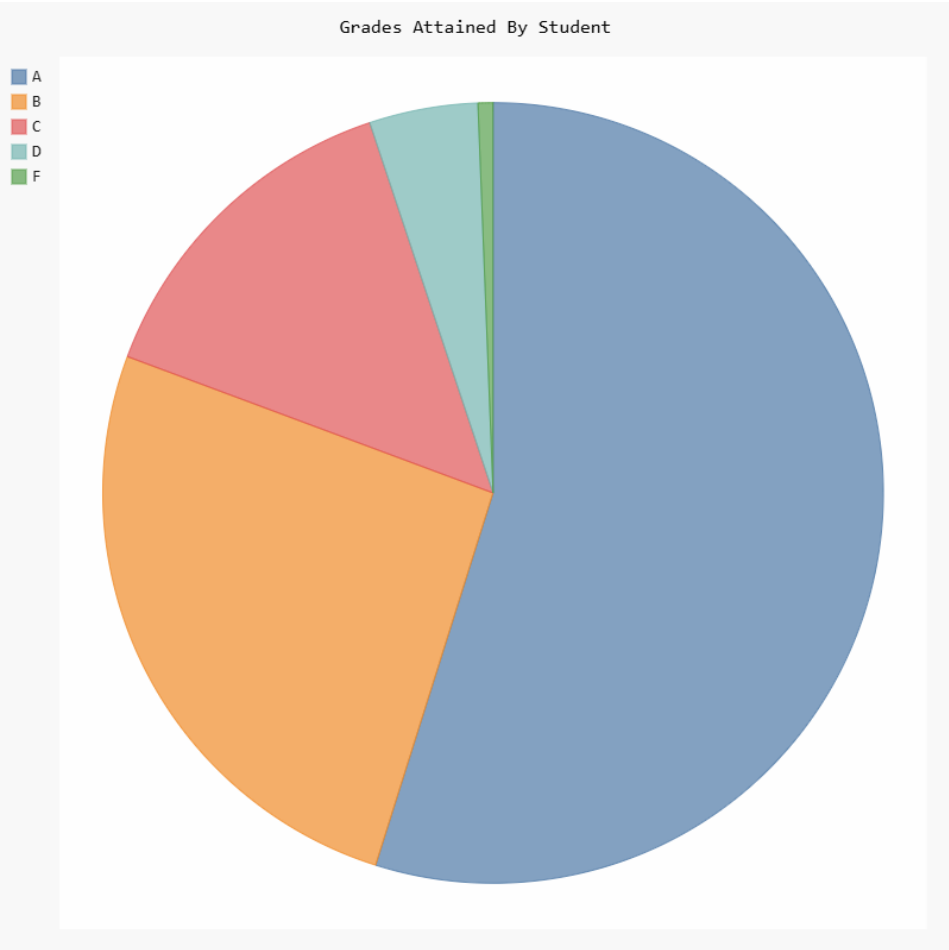


Summary:

The above data is the composite data from a university's attendance logging app. It shows the Attendance Rating (a metric for determining how many classes of his / her total a student attends). Since the data is stored as a value rounded to the nearest half, we can get explicit frequencies for each

attendance rating. The frequencies for each attendance rating are first recorded and compiled into a frequency table. This frequency table is as pasted above. It provides the raw numerical values for each attendance rating. From this, using the Pygal library (a library to create .svg files to better show off data), we can create a Bar Diagram and its corresponding Frequency Polygon to better understand the data visually. As we can see, the key features of the data include an almost normal-distribution like curve for attendance ratings, with the most common attendance rating being a 6.0 (the mode). We may also observe the spike at 10, showing that many students are likely to make an extra effort to have a perfect attendance rating when they are already very close to it. From the frequency polygon, we observe a growing trend of attendance rating (more people are likely to have a higher attendance rating) until the modal value, where it then drops off and a decreasing trend of attendance values is obtained instead.

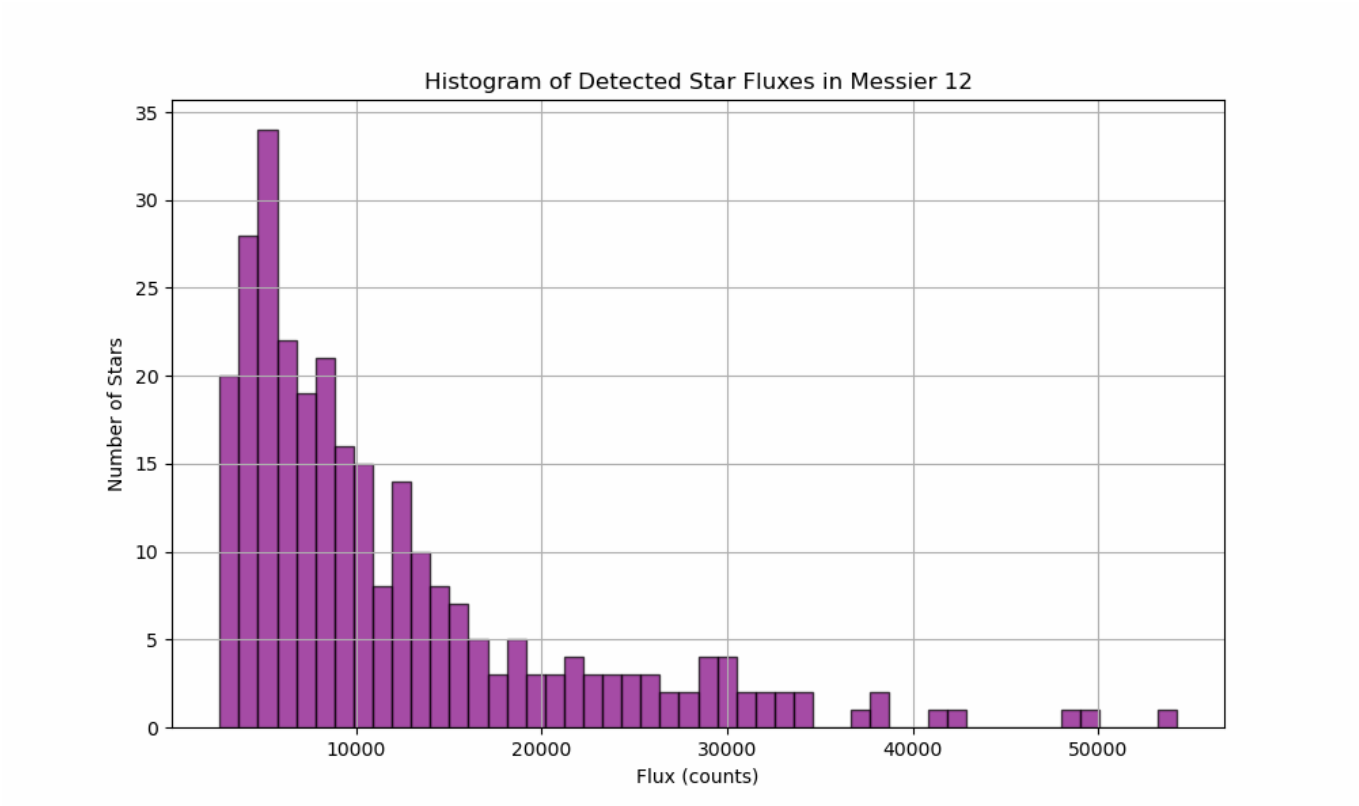
Pie Chart:



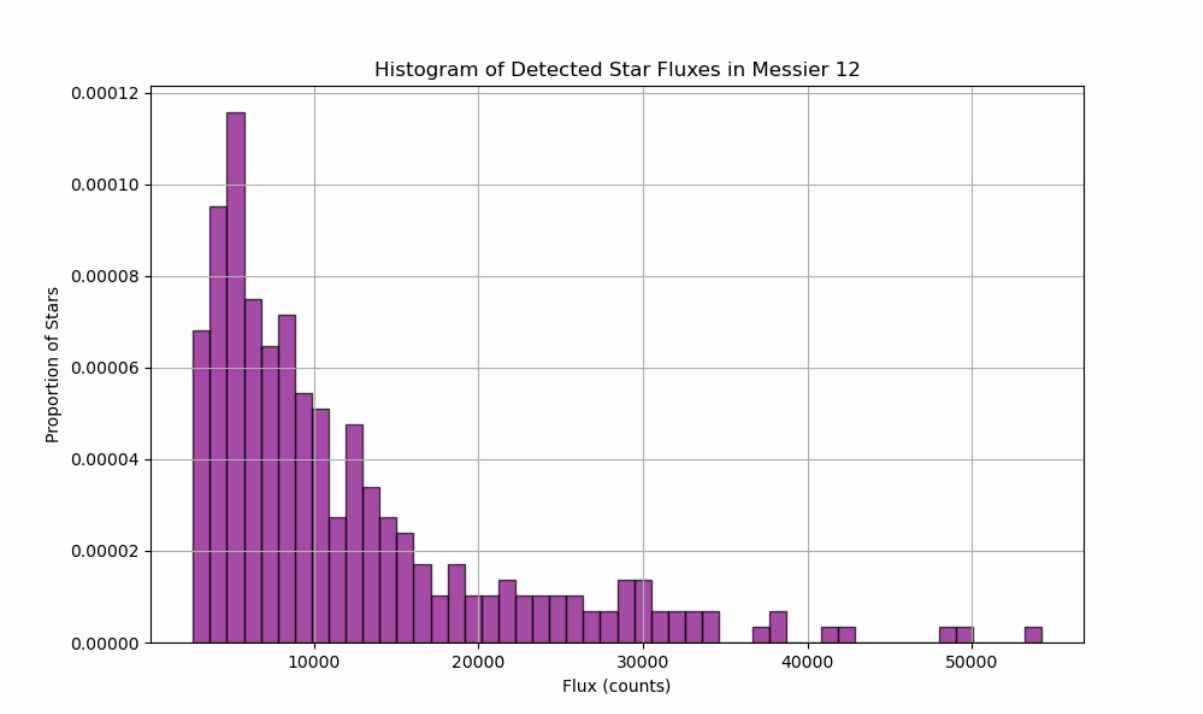
Summary:

A pie chart is a useful, visually intuitive data analysis tool which divides each category or possibility into slices according to its relative frequency. To construct a pie chart, we calculate the frequency of the occurrence of each possibility. Then, we divide each frequency by the total number of data points collected (thus obtaining the relative frequency). This is again fed to a library function in Pygal and we get the above Pie Chart. From this, we can easily extract information about what the average, consensus, or majority representation looks like. We can also at a glance get an idea of what the least common preferences or results are. It represents all the data proportionally. Here, we have taken the grade data of a course and created a pie chart of it. From this, we may determine, as a student, that the course is particularly easy as many people have been awarded an A or B, and very few people have failed. The instructor may draw the conclusion that they should increase the difficulty of the course to give more fair grading to everyone.

Histogram:



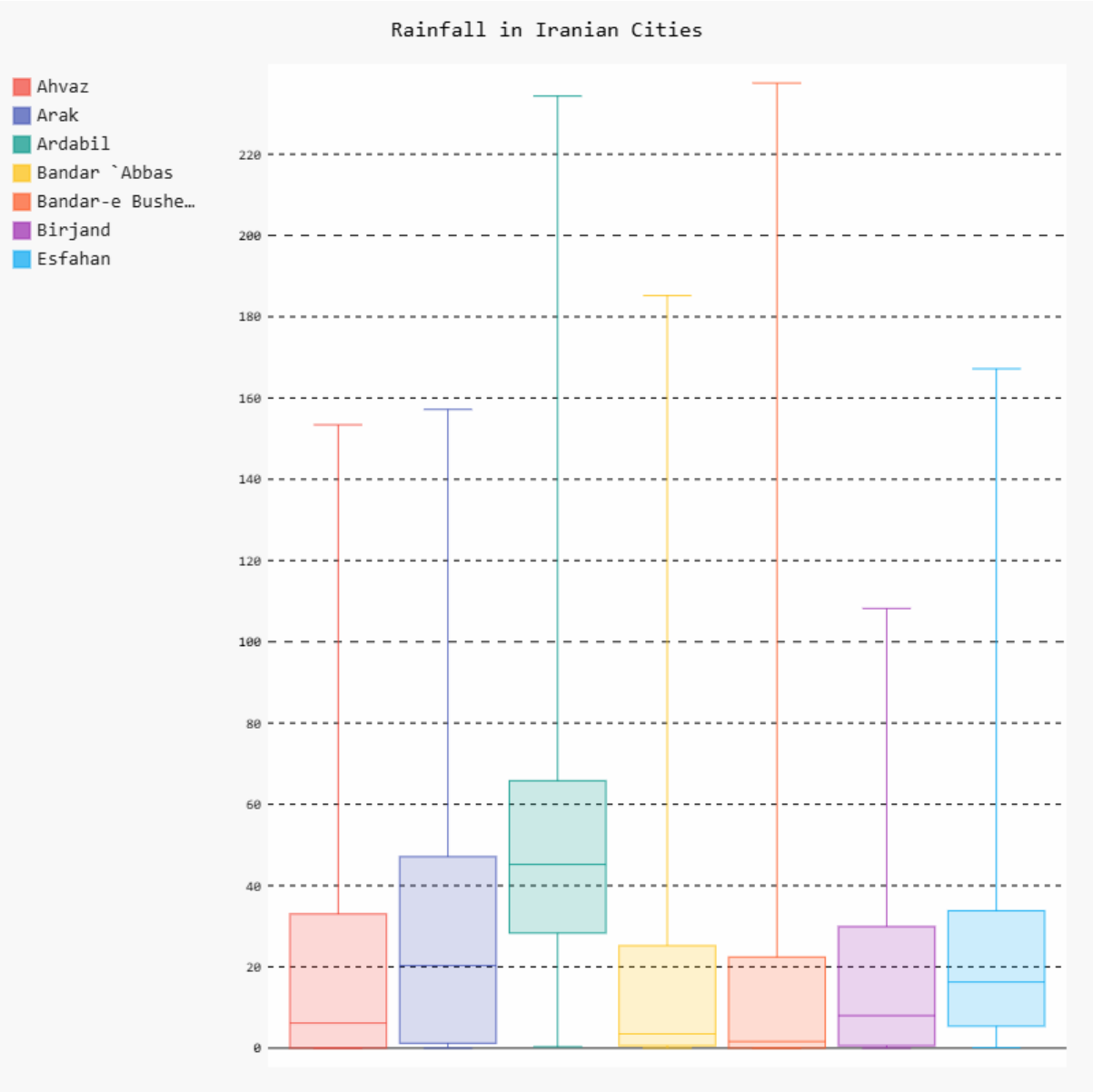
Relative Histogram:



Summary:

For the histograms, I have used the in-built functionality of the MatPlotLib library. The histogram itself represents the net flux detected of the stars in the Messier 12 Global Cluster (a star cluster located in the Milky Way galaxy). The extraction of the data from .fits file (which is a common way of storing image data in astronomy) is left out of this discussion, but the basic premise is that we have identified the stars in the image and collected the net flux detected from each one of them. Since this data is nearly continuous (limited only to the precision of the measuring instrument), we divide it into 50 bins and appropriately construct the histogram. Scaling this by the total number of stars detected, we can also obtain the relative frequency histogram. From this, we can draw similar conclusions to those we could draw about discrete data as above. Most of the stars have a flux between three and nine thousand counts. There are outliers of stars which have nearly 50 thousand counts.

Box Plot:



Summary:

The data of rainfalls in Iranian cities from 1901 to 2022 in the given 7 cities is collected. As we can see, there are very large outliers where each city experienced abnormally high rainfall. Since most cities experience very little rainfall, their minimum values are not visible. However, for ‘Ardabil’, we can observe that the lowest recorded rainfall is zero although it usually experiences a good bit more. The data is also split into quartiles inside the boxes with the line in the middle of the box giving the median. For such large datasets with exorbitant outliers, the median makes the most sense as a measure of central tendency, and such a box plot allows us to easily, at a glance compare which

cities receive more rainfall and which do not receive as much. A farming company may use this data visualization and conclude that all these cities are receiving too little rainfall to be profitable ventures. We may also guess that crops which are more suited to arid conditions are more commonly grown in Iran due to their low expected rainfall (where the median and the range of the first to third quartiles is being used as a stand-in for the expectation value).

Python Codes:

Points 1, 2, 3:

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from pygal.style import Style
import pygal
import csv

filename = 'student_performance.csv'

with open(filename) as f:
    reader = csv.reader(f)
    header_row = next(reader)
    # print(header_row)
    # for index, column_header in enumerate(header_row):
    #     print(index, column_header)
    participation = []
    grades = []
    for row in reader:
        high = float(row[6])
        participation.append(high)
        grade = row[5].strip()[0]
        grades.append(grade)
```

```
#constructing the frequency table
labels = []
frequencies = []
for value in np.arange(0, 10.5, 0.5):
    labels.append(value)
    frequency = participation.count(value)
    frequencies.append(frequency)

table_data = [[label, freq] for label, freq in zip(labels, frequencies)]
fig, ax = plt.subplots(figsize=(6, len(labels) * 0.35))
ax.axis('off')

table = ax.table(
    cellText=table_data,
    colLabels=["Attendance Rating", "Frequency"],
    loc="center",
    cellLoc="center"
)

table.auto_set_font_size(False)
table.set_fontsize(10)
table.scale(1, 1.4)

plt.savefig("frequency_table.svg", bbox_inches="tight", dpi=200)
plt.close()
```

```
freq_table = pygal.Bar(width=800, height=400, explicit_size=True)
freq_table.title = "Class Attendance Ratings of Students"
freq_table.x_labels = map(str, np.arange(0, 10.5, 0.5))
freq_table.add('Attendance Rating', frequencies)
freq_table.render_to_file('bar_chart.svg')

freq_table = pygal.Line(width=800, height=800, explicit_size=True)
freq_table.title = "Class Attendance Ratings of Students"
freq_table.x_labels = map(str, np.arange(0.25, 10.5, 0.5))
freq_table.add('Attendance Rating', frequencies)
freq_table.render_to_file('line_chart.svg')

labels = ['A', 'B', 'C', 'D', 'F']
grade_freq = []
total = 0
for value in labels:
    frequency = grades.count(value)
    total += frequency
    grade_freq.append(frequency)

custom_style = Style(
    colors=(
        '#4e79a7',
        '#f28e2b',
        '#e15759',
        '#76b7b2',
```

```
        '#59a14f'
    )
)
pie_chart = pygal.Pie(width=800, height=800, explicit_size=True,
style=custom_style)
pie_chart.title = "Grades Attained By Student"
for i in range(0, 5):
    pie_chart.add(labels[i], (grade_freq[i] * 100.00)/(total))
pie_chart.render_to_file('pie_chart.svg')
```

Point 4:

```
import numpy as np
import matplotlib.pyplot as plt
import astropy.io.fits
import astropy.stats
from astropy.stats import sigma_clipped_stats
from astropy.stats import sigma_clip
from photutils.detection import DAOStarFinder
from photutils.aperture import CircularAperture
from photutils.aperture import aperture_photometry

#Loading in the fits file
fits = astropy.io.fits.open("Vcomb.fits")
picData = fits[0].data
fits.close()
```

```
#checking for outlier data using sigma clipping
mean, median, std = astropy.stats.sigma_clipped_stats(picData, sigma_lower = 2,
sigma_upper = 5)
clippedData = sigma_clip(picData, sigma_lower = 1, sigma_upper = 20)
no_outliers = np.sum(clippedData.mask)

#Using DAOStarFinder to detect stars, and highlighting them with the circular aperture
daofind = DAOStarFinder(fwhm = 20, threshold = 5 * std)
sources = daofind(clippedData - median)
positions = np.transpose((sources['xcentroid'], sources['ycentroid']))
apertures = CircularAperture(positions, r = 10)
#finding flux in the star apertures encircled above using aperture photometry
phot_table = aperture_photometry(clippedData, apertures)
fluxes = phot_table['aperture_sum']

# Plot histogram of fluxes
plt.figure(figsize=(10, 6))
plt.hist(fluxes, bins=50, color='purple', alpha=0.7, edgecolor='black', density = False)
plt.xlabel('Flux (counts)')
```

```
plt.ylabel('Number of Stars')
plt.title('Histogram of Detected Star Fluxes in Messier 12')
plt.grid(True)
plt.show()

# Plot histogram of relative fluxes
plt.figure(figsize=(10, 6))
plt.hist(fluxes, bins=50, color='purple', alpha=0.7, edgecolor='black', density = True)
plt.xlabel('Flux (counts)')
plt.ylabel('Number of Stars')
plt.title('Histogram of Detected Star Fluxes in Messier 12')
plt.grid(True)
plt.show()
```

Point 5:

```
import matplotlib.pyplot as plt
import pygal
import csv

filename = 'Rainfall_Iran_19012022.csv'

c1 = []
c2 = []
c3 = []
```

```
c4 = []
c5 = []
c6 = []
c7 = []

with open(filename) as f:
    reader = csv.reader(f)
    header_row = next(reader)
    # print(header_row)
    # for index, column_header in enumerate(header_row):
    #     print(index, column_header)
    for row in reader:
        high = float(row[1])
        c1.append(high)
        high = float(row[2])
        c2.append(high)
        high = float(row[3])
        c3.append(high)
        high = float(row[4])
        c4.append(high)
        high = float(row[5])
        c5.append(high)
        high = float(row[6])
        c6.append(high)
        high = float(row[7])
        c7.append(high)
```

```
labels = ["Ahvaz", "Arak", "Ardabil", "Bandar `Abbas", "Bandar-e Bushehr",
"Birjand", "Esfahan"]


box_plot = pygal.Box(width=800, height=800, explicit_size=True)
box_plot.title = 'Rainfall in Iranian Cities'
box_plot.add(labels[0], c1)
box_plot.add(labels[1], c2)
box_plot.add(labels[2], c3)
box_plot.add(labels[3], c4)
box_plot.add(labels[4], c5)
box_plot.add(labels[5], c6)
box_plot.add(labels[6], c7)
box_plot.render()
box_plot.render_to_file('box_plot.svg')
```